Altoro Mutual Bank Penetration Testing Report

Prepared by:

SAMSUZZOHA SHAMIM
Software Security Engineer | Analyst

Date: 10-05-2026

Professional Summary

Dedicated Software Security Engineer with over 2 years of hands-on experience in Web and Mobile Application Security Testing, Vulnerability Assessment, and Penetration Testing (VAPT). Specialized in identifying, analyzing, and validating security vulnerabilities across modern applications using both automated and manual testing methodologies. Strong expertise in conducting security assessments based on OWASP Top 10 standards, industry best practices, and risk-based testing approaches to strengthen overall application security posture.

Experienced in performing comprehensive security testing for web applications, mobile applications, and APIs, including authentication and authorization testing, session management analysis, input validation testing, business logic testing, security misconfiguration assessment, and API security validation. Skilled in discovering vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection, IDOR, Authentication Bypass, Security Misconfigurations, Sensitive Data Exposure, CSRF, SSRF, and other application-layer security flaws. Proficient in using industry-standard security tools including Burp Suite, OWASP ZAP, and various reconnaissance and testing utilities for effective vulnerability identification and validation.

Strong experience in preparing professional security assessment reports with detailed proof-of-concept findings, risk ratings, impact analysis, and remediation recommendations to support secure development practices. Capable of communicating technical vulnerabilities clearly to development and engineering teams while assisting in vulnerability remediation and retesting activities to ensure effective issue resolution.

Additionally, bringing more than 4 years of experience in Software Quality Assurance (SQA) and software testing, which provides a strong foundation in application behavior analysis, test planning, defect identification, and overall software quality processes. This combined background in security testing and quality assurance enables a deeper understanding of application workflows, functionality, and risk areas, contributing to more effective and thorough security assessments.

Passionate about application security, ethical hacking, and continuous improvement in cybersecurity practices. Focused on helping organizations build secure, reliable, and resilient applications through proactive security testing and risk-driven assessment methodologies.

Table of Contents

Contents

Table of Contents 3
1 Web Application Penetration Testing 4
1.1 Executive Summary 4
1.2 Background 4
1.3 Objectives 4
1.4 Scope of Assessment 5
1.5 Out of Scope 5
1.6 Performed Test 5
1.7 Used Tools 6
1.7 Summary of the Penetration Testing 7
2 Exploitation Details 9
2.1 SQL Injection 9
2.2 XSS (Cross Site Scripting) 11
2.3 Brute Force Attacks 14
2.4 Cross-Site Request Forgery (CSRF) 16
2.5 No Rate Limit 17
2.6 Open Redirection (DOM-based) 18
2.7 Frameable Response (Potential Clickjacking) 19
2.8 Content Security Policy (CSP) Issues 21
2.9 Strict Transport Security (HSTS) Not Enforced 22
3 Informational Issues and Remediation Suggestion 24
4.0 Testing Methodology 28
5.0 Testing Methodology 30
6.0 Conclusion 31

1 Web Application Penetration Testing

1.1 Executive Summary

The purpose of these Web Security Assessments was to identify and exploit exploitable vulnerabilities and insufficiently configured security controls to determine the likelihood that users with considerable, little, or no prior knowledge of the applications (e.g., informed, and uninformed insiders and outsiders) could exploit weaknesses in their applications as those cataloged in the OWASP Top 10, OWASP ASVS, SANS, NIST, OWASP testing guide and Penetration Testing Execution Standard.

The team leveraged tools to facilitate their work, however, the majority of the assessment involved automation & manual analysis. The details of the tools used in the testing are given in this report.

The results summarized in this document are based upon a collection of methodologies and tests interacting at a single point in time with technology that is continually changing and becoming ever more complex. Any projection to the future of the findings contained in this document is subject to the risk that they may no longer portray the system or environment in existence at that time because of change.

1.2 Background

This exercise aimed to perform Penetration Testing of the Applications in scope to determine if they were vulnerable to attacks and exploitation. The test consisted of automation & manual testing to detect and exploit vulnerabilities.

The assessment was conducted to identify and exploit the security vulnerabilities in the Application in scope and propose solutions for the project team to remediate the identified vulnerabilities to make the Application more secure.

The Security Assessment was performed by **Samsuzzoha Shamim** between **1st May 2026** to

10th May 2026.

1.3 Objectives

The objective of the tests performed was to:

- Exploit the possible vulnerabilities and gaps in the web applications.
- Assess the gaps and vulnerabilities to determine the Risk associated.
- Suggest recommendations to overcome existing vulnerabilities and gaps.

This assessment report contains:

- Technical details of the identified vulnerabilities, including supporting evidence and validation of the findings.
- Risk mitigation recommendations and remediation measures required to reduce the security risks associated with the identified vulnerabilities and improve the overall security posture of the systems.

1.4 Scope of Assessment

URLs

- https://demo.testfire.net

Accounts

User	Role	Password
jsmith	User	Demo1234

Environment Details

Application Name	Altoro Mutual Bank
Environment	Apache-Coyote/1.1
Accessibility	Internet
Authentication Method	Login
Frontend	Java/J2EE
Backend	Java/J2EE
Server	Apache-Coyote/1.1

1.5 Out of Scope

The following activities and applications were considered out of scope for this Engagement:

- Social Engineering
- Vulnerability Fixes
- Network Penetration Testing.

1.6 Performed Test

- SQL Injection
- Cross-Site Scripting (XSS)
- Brute Force Attacks
- Cross-Site Request Forgery (CSRF)
- No Rate Limit
- Open Redirection (DOM-based)
- Frameable Response (Potential Clickjacking)
- Content Security Policy Not Implemented
- Strict Transport Security (HSTS) Not Enforced
- Cacheable HTTPS Response
- DOM Data Manipulation (DOM-based)
- Cross-domain Referrer Leakage
- Path-relative Style Sheet Import
- Referrer-dependent Response
- Input Returned in Response (Reflected)
- Sensitive Data Exposure (Email Address)
- HTML Does Not Specify Charset
- TLS Certificate Observation

1.7 Used Tools

Commlink Info Tech ltd penetration testing team uses a combination of automation & manual testing and commercial/open-source tools as part of its penetration testing methodology. This is accompanied by custom scripts to ensure optimum results.

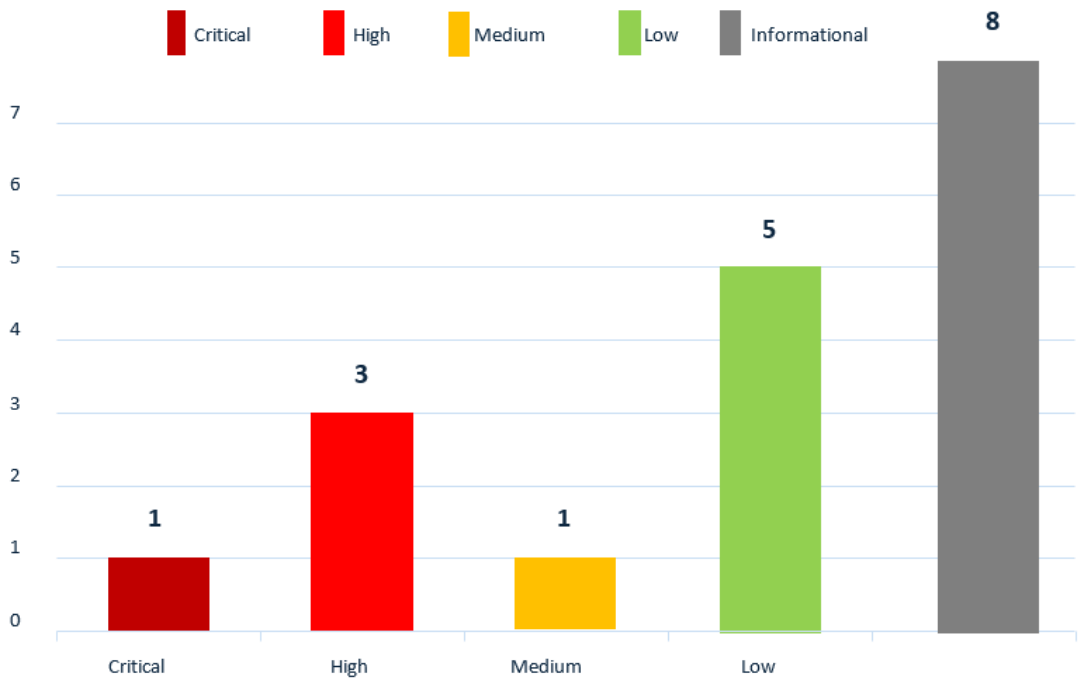
The following tools are used for the testing.

- Kali Linux
- Burp Suite Pro
- Acunetix
- SQL Map
- jSQL
- Metasploit
- MSF Venom
- Katana
- Dalfox
- Browser Dev Tool

1.7 Summary of the Penetration Testing

Following the Vulnerability Assessment (VA) conducted during the previous assessment phase, a Web Penetration Testing (PT) exercise was performed to validate the identified findings and assess their potential impact on the target application. The assessment identified multiple security weaknesses and configuration related issues affecting the overall security posture of the application. Overall, the testing highlighted areas requiring remediation to reduce potential security risks and improve resilience against cyber threats.

Sl. No.	Name	Findings	Risk Score	Severity
1	SQL Injection	Yes	9.1	Critical
2	XSS (Cross Site Scripting)	Yes	8.2	High
3	Brute Force Attacks	Yes	8.1	High
4	Cross-Site Request Forgery (CSRF)	Yes	8.0	High
5	No Rate Limit	Yes	6.5	Medium
6	Open Redirection (DOM-based)	Yes	4.3	Low
7	Frameable Response (Potential Clickjacking)	Yes	4.0	Low
8	Content Secure Policy	Yes	4.0	Low
9	Strict Transport Security (HSTS) Not Enforced	Yes	3.9	Low
11	Cacheable HTTPS Response	Yes	3.1	Informational
12	DOM Data Manipulation (DOM-based)	Yes	2.8	Informational
13	Cross-domain Referrer Leakage	Yes	2.6	Informational
14	Path-relative Style Sheet Import	Yes	2.3	Informational
15	Referrer-dependent Response	Yes	2.1	Informational
16	Input Returned in Response (Reflected)	Yes	2.0	Informational
17	Sensitive Data Exposure (Email Address)	Yes	1.8	Informational
18	HTML Does Not Specify Charset	Yes	1.0	Informational
19	TLS Certificate Observation	Yes	1.0	Informational



During the vulnerability assessment, a total of 19 security findings were identified within the application. The findings include 1 Critical, 3 High, 1 Medium, 5 Low, and 9 Informational vulnerabilities.

The identified vulnerabilities indicate the presence of security weaknesses related to input validation, authentication protection, security misconfigurations, and missing security controls. These issues may increase the risk of unauthorized access, client-side attacks, and information disclosure if not properly addressed.

It is recommended that the identified vulnerabilities be remediated based on their severity level to improve the overall security posture of the application.

2 Exploitation Details

2.1 SQL Injection

SQL Injection is a critical vulnerability that occurs when untrusted user input is improperly processed within SQL queries. Successful exploitation may allow attackers to manipulate database queries, access sensitive data, bypass authentication mechanisms, or compromise the backend database.

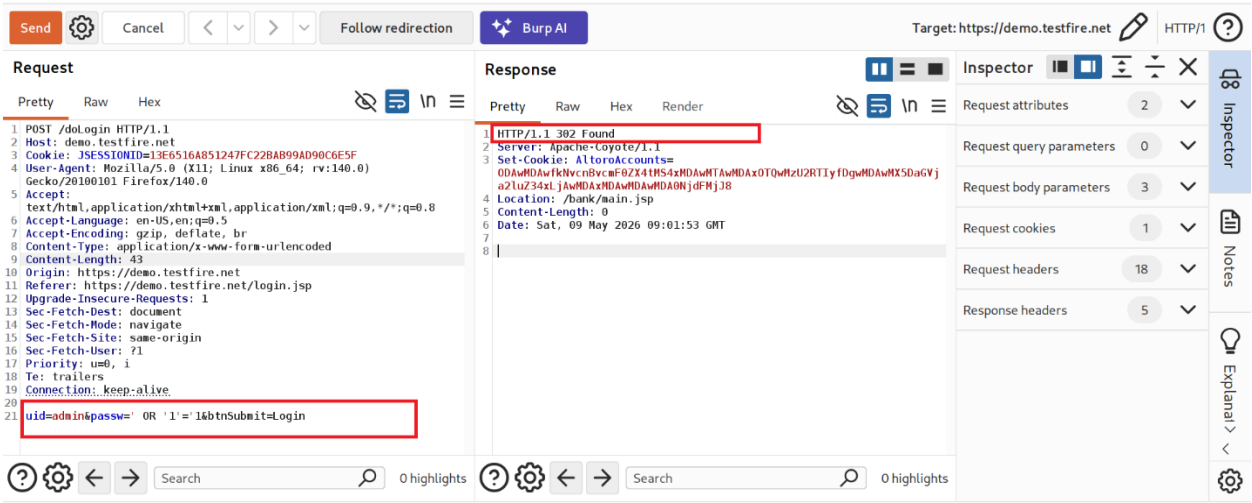
Exploitation Details

During the penetration testing assessment, it was identified that the application does not properly validate or sanitize user-supplied input before processing database queries. This weakness may allow malicious SQL statements to be executed against the backend database.

Used Tools

- Burp Suite Pro
- Acunetix
- SQLMap
- JSQL
- OWASP Zap

Proof of Concept Screenshot



Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Discover

1 | 2

Cluster bomb attack | Start attack

Target https://demo.testfire.net | Update Host header to match target

Positions | Add § | Clear § | Auto §

```
1  POST /doLogin HTTP/1.1
2  Host: demo.testfire.net
3  Cookie: JSESSIONID=13E6516A851247FC22BAB99AD90C6E5F
4  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6  Accept-Language: en-US,en;q=0.5
7  Accept-Encoding: gzip, deflate, br
8  Content-Type: application/x-www-form-urlencoded
9  Content-Length: 48
10 Origin: https://demo.testfire.net
11 Referer: https://demo.testfire.net/login.jsp
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 uid=§test%40gmail.com§&passw=§12345§&btnSubmit=Login
```

Search | 2 highlights | 2 payload positions | Length: 719

Payloads

Payload position: 1 - test%40gmail.com
Payload type: Simple list
Payload count: 114
Request count: 12,996

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste | Load... | Remove | Clear | Deduplicate | Add

```
'
"
'--
'#
'/*
' OR '1'='1
' OR 1=1--
```

Enter a new item

Add from list...

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Payloads | Resource pool | Settings

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Discover

1 | 2

Cluster bomb attack | Start attack

Target https://demo.testfire.net | Update Host header to match target

Positions | Add § | Clear § | Auto §

```
1  POST /doLogin HTTP/1.1
2  Host: demo.testfire.net
3  Cookie: JSESSIONID=13E6516A851247FC22BAB99AD90C6E5F
4  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6  Accept-Language: en-US,en;q=0.5
7  Accept-Encoding: gzip, deflate, br
8  Content-Type: application/x-www-form-urlencoded
9  Content-Length: 48
10 Origin: https://demo.testfire.net
11 Referer: https://demo.testfire.net/login.jsp
12 Upgrade-Insecure-Requests: 1
13 Sec-Fetch-Dest: document
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-User: ?1
17 Priority: u=0, i
18 Te: trailers
19 Connection: keep-alive
20
21 uid=§test%40gmail.com§&passw=§12345§&btnSubmit=Login
```

Search | 2 highlights | 2 payload positions | Length: 719

Payloads

Payload position: 2 - 12345
Payload type: Simple list
Payload count: 114
Request count: 12,996

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste | Load... | Remove | Clear | Deduplicate | Add

```
' ORDER BY 10--
'); WAITFOR DELAY '0:0:5'--
'); SELECT PG_SLEEP(5)--
' OR (SELECT 1 FROM (SELECT(SLEEP(5)))a)='
' OR (SELECT 1 FROM (SELECT(SLEEP(5)))a)...
1' AND 1=1
1' AND 1=2
```

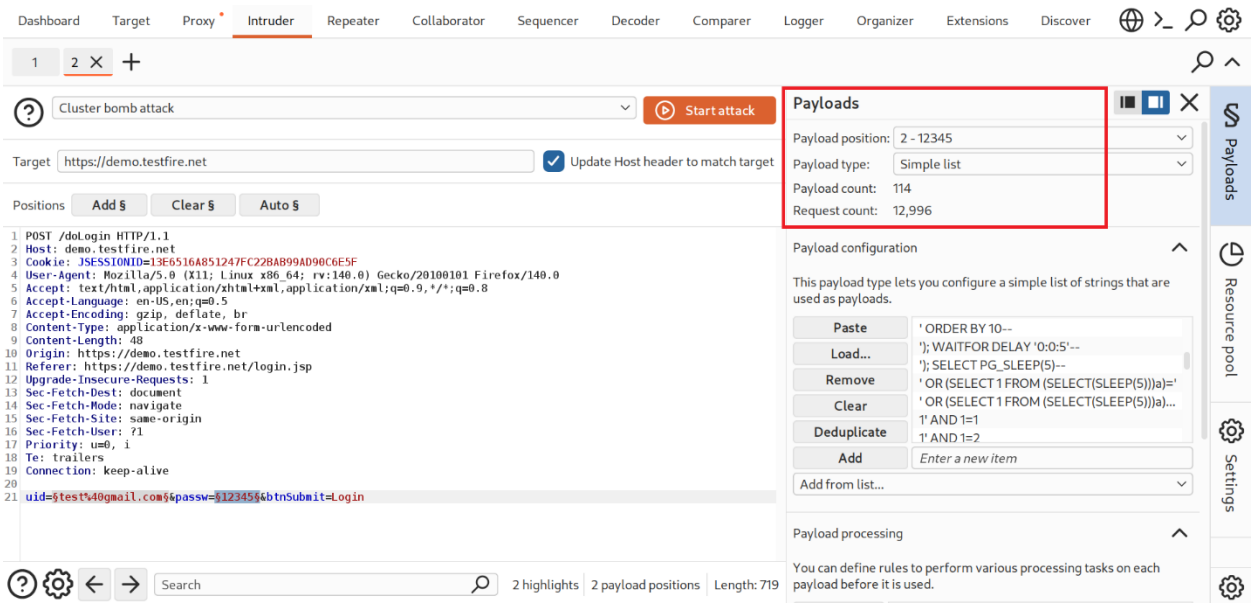
Enter a new item

Add from list...

Payload processing

You can define rules to perform various processing tasks on each payload before it is used.

Payloads | Resource pool | Settings



Resolution

- Use parameterized queries or prepared statements for all database interactions.
- Implement strict server-side input validation and sanitization.
- Avoid dynamic SQL query construction using user-controlled input.
- Apply the principle of least privilege for database accounts.
- Implement proper error handling to prevent database error disclosure.

2.2 XSS (Cross Site Scripting)

Cross-Site Scripting (XSS) is a client-side vulnerability that occurs when user-controlled input is improperly handled and executed within a web browser. Successful exploitation may allow attackers to execute malicious scripts, hijack user sessions, manipulate web content, or perform actions on behalf of legitimate users.

Exploitation Details

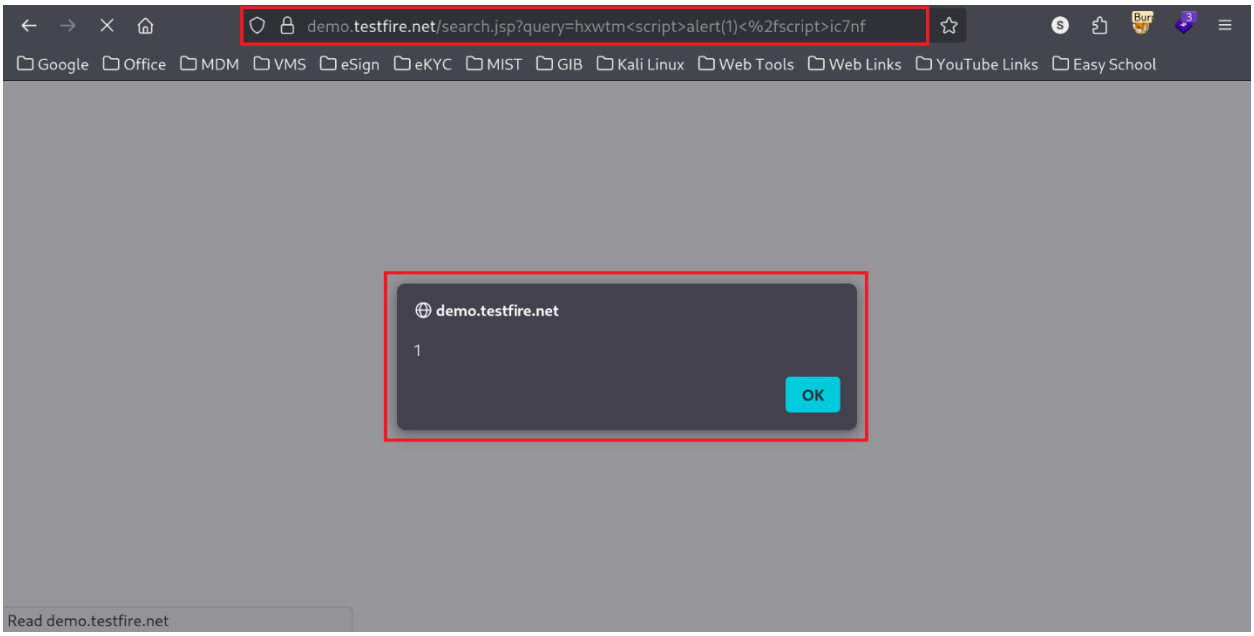
During the penetration testing assessment, it was identified that the application does not properly validate or sanitize user-supplied input before rendering it within the browser. This weakness may allow malicious scripts to be injected and executed in the context of the victim's browser session.

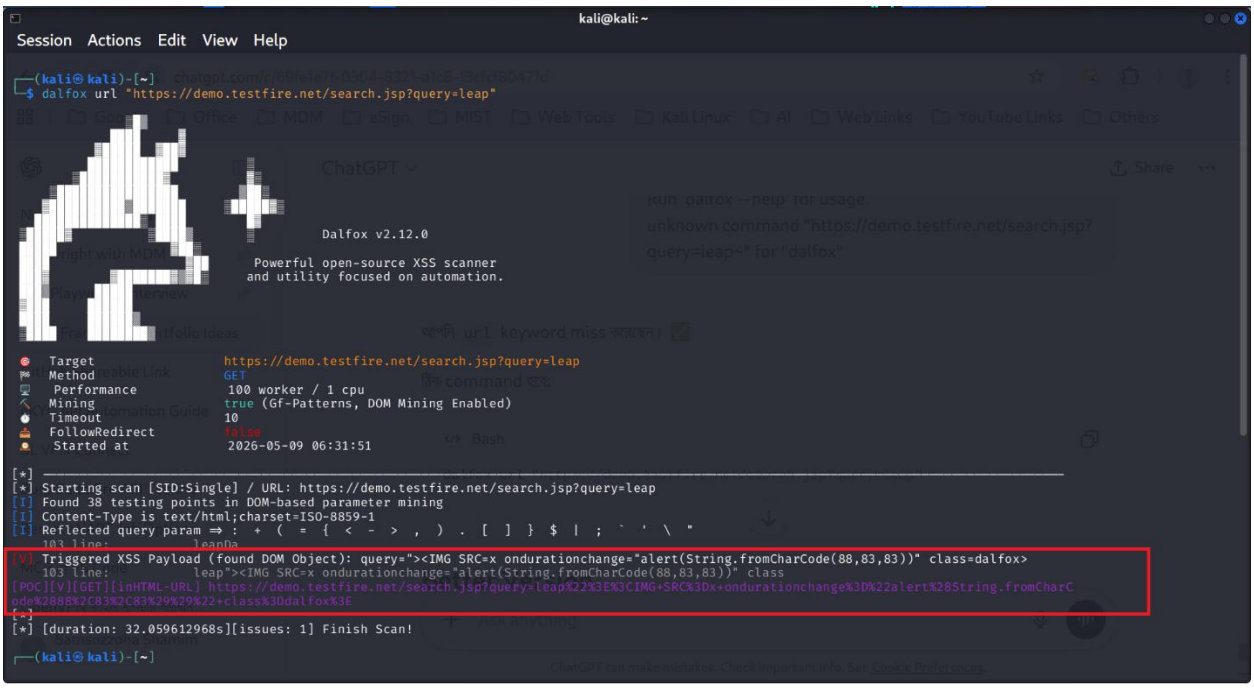
Used Tools

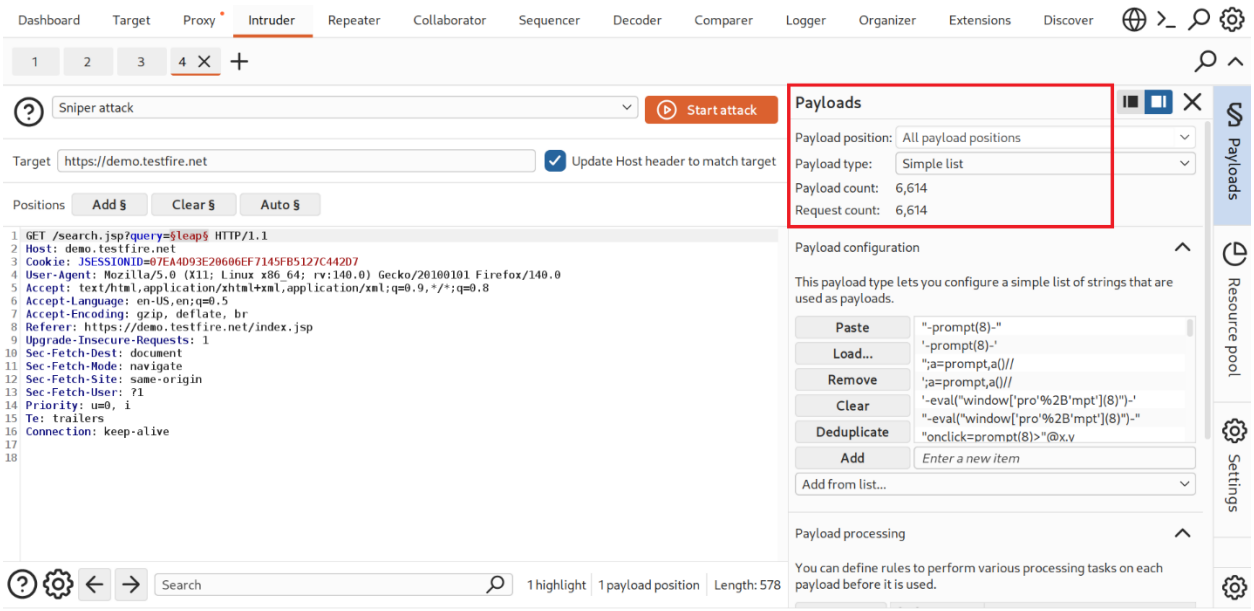
- Burp Suite Pro
- Acunetix
- SQLMap

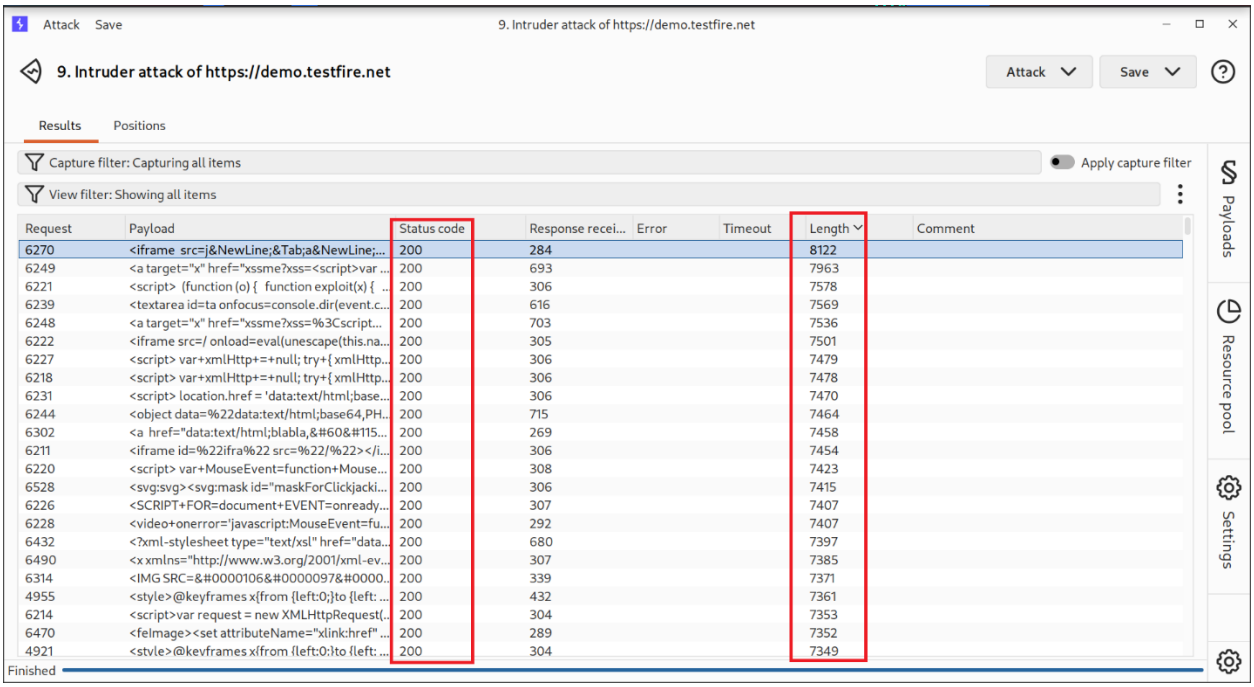
- Katana
- Dalfox

Proof of Concept Screenshot









Resolution

- Implement proper input validation and output encoding.
- Sanitize all user-supplied input before rendering it in the browser.
- Apply Content Security Policy (CSP) headers.
- Avoid directly inserting user input into HTML, JavaScript, or DOM elements.
- Use secure frameworks that automatically escape output content.

2.3 Brute Force Attacks

When Content Security Policy (CSP) is not implemented, the web application lacks a mechanism to control and restrict the sources from which it can load resources, such as scripts, stylesheets, or images. This absence of CSP opens the site to vulnerabilities, as it becomes more susceptible to malicious scripts or content injected by attackers.

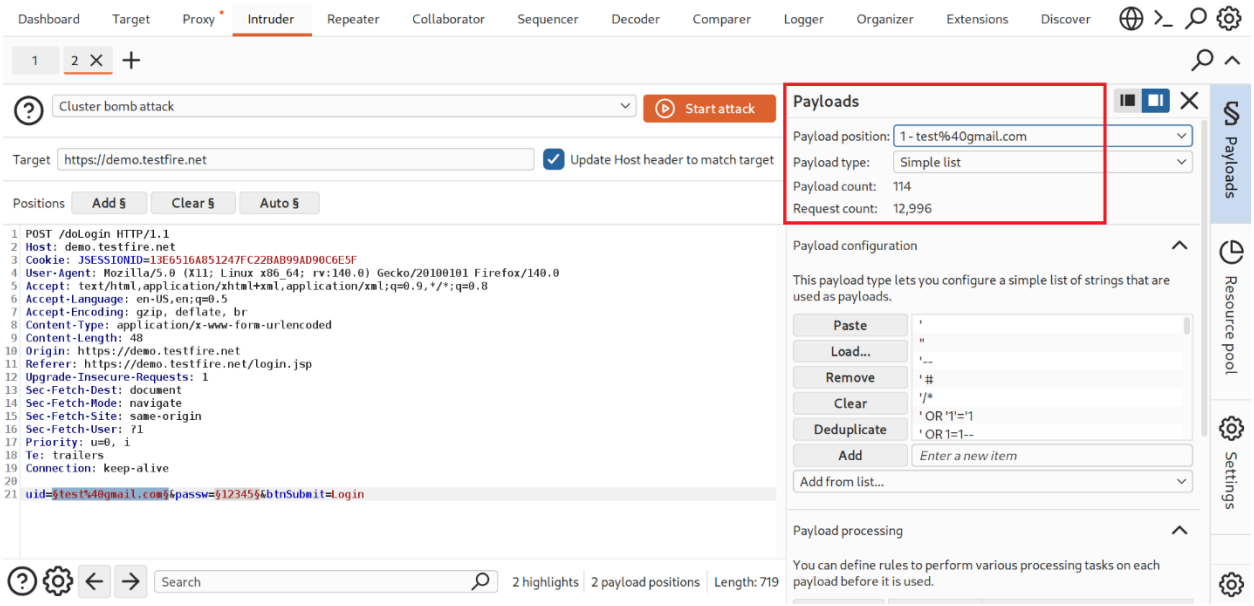
Exploitation Details:

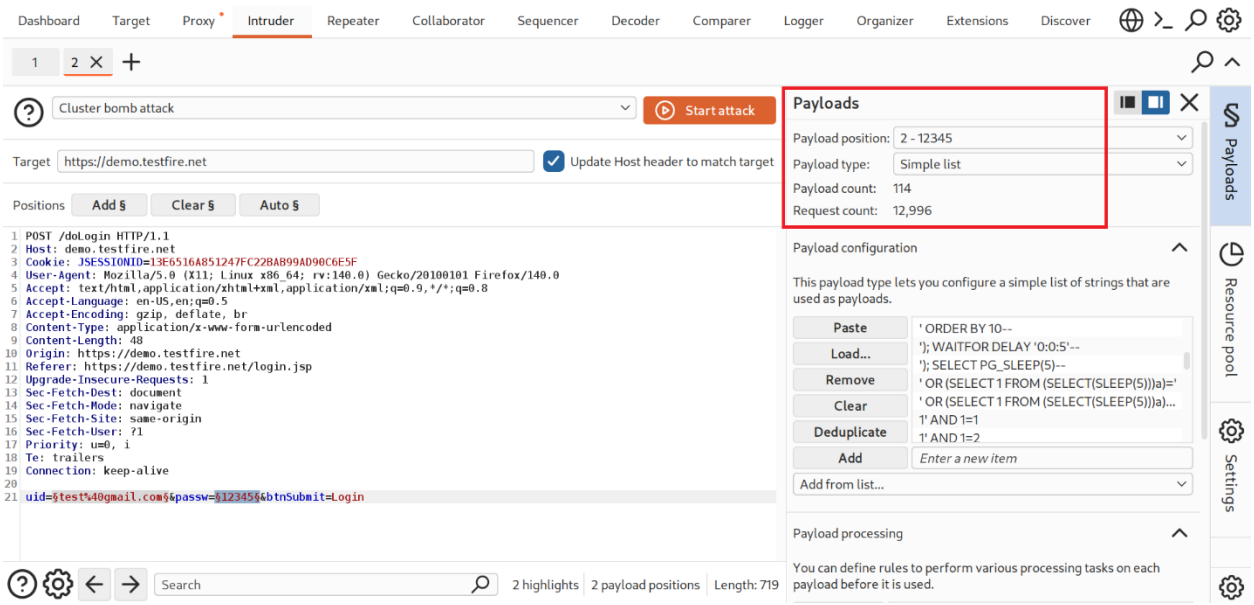
During the penetration testing assessment, it was identified that the application does not properly restrict repeated authentication attempts. This weakness may allow attackers to perform automated password guessing attacks against valid user accounts.

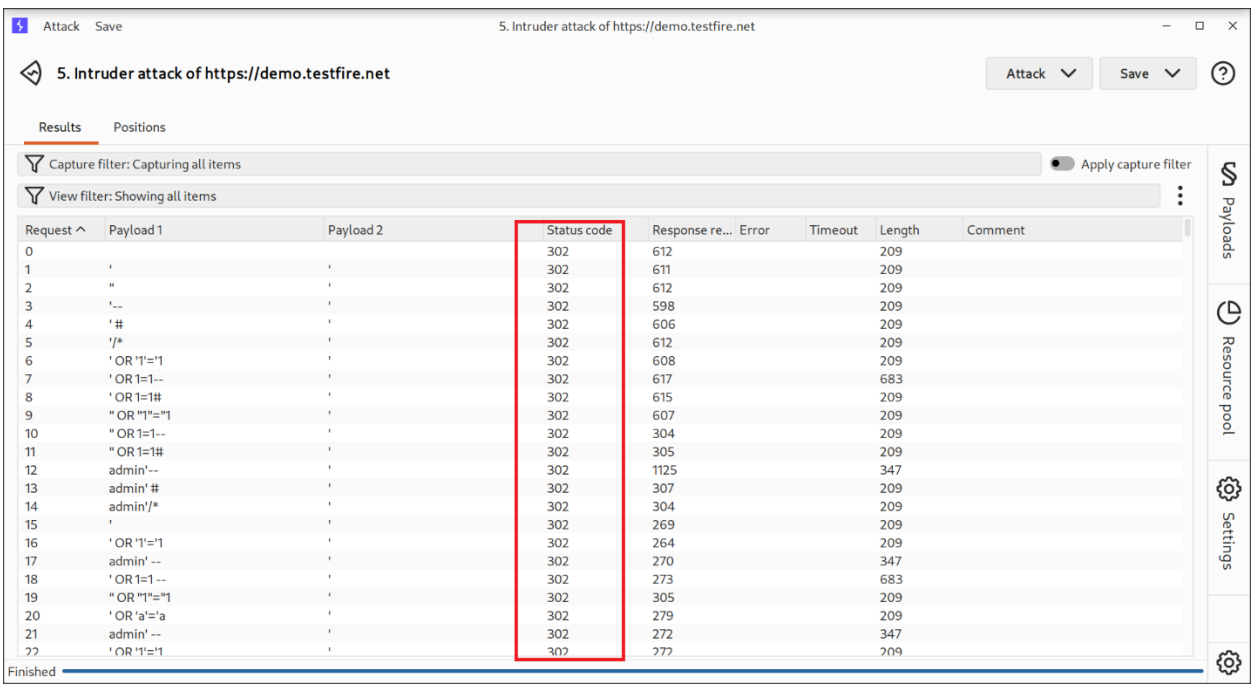
Used Tools

- Burp Suite Pro
- Hydra
- OWASP ZAP

Proof of Concept Screenshot







Request	Payload 1	Payload 2	Status code	Response re...	Error	Timeout	Length	Comment
0			302	612			209	
1	'	'	302	611			209	
2	"	'	302	612			209	
3	'--	'	302	598			209	
4	'#	'	302	606			209	
5	'/*	'	302	612			209	
6	' OR '1'='1	'	302	608			209	
7	' OR 1=1--	'	302	617			683	
8	' OR 1=1#	'	302	615			209	
9	" OR "1"="1	'	302	607			209	
10	" OR 1=1--	'	302	304			209	
11	" OR 1=1#	'	302	305			209	
12	admin'--	'	302	1125			347	
13	admin' #	'	302	307			209	
14	admin'/*	'	302	304			209	
15	'	'	302	269			209	
16	' OR '1'='1	'	302	264			209	
17	admin' --	'	302	270			347	
18	' OR 1=1 --	'	302	273			683	
19	" OR "1"="1	'	302	305			209	
20	' OR 'a'='a	'	302	279			209	
21	admin' --	'	302	272			347	
22	' OR '1'='1	'	302	272			209	

Resolution

- Implement account lockout mechanisms after multiple failed login attempts.
- Apply rate limiting and CAPTCHA protections on authentication endpoints.
- Enforce strong password policies.
- Enable multi-factor authentication (MFA).
- Monitor and log suspicious authentication activities.

2.4 Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a vulnerability that forces authenticated users to perform unintended actions on a web application without their consent.

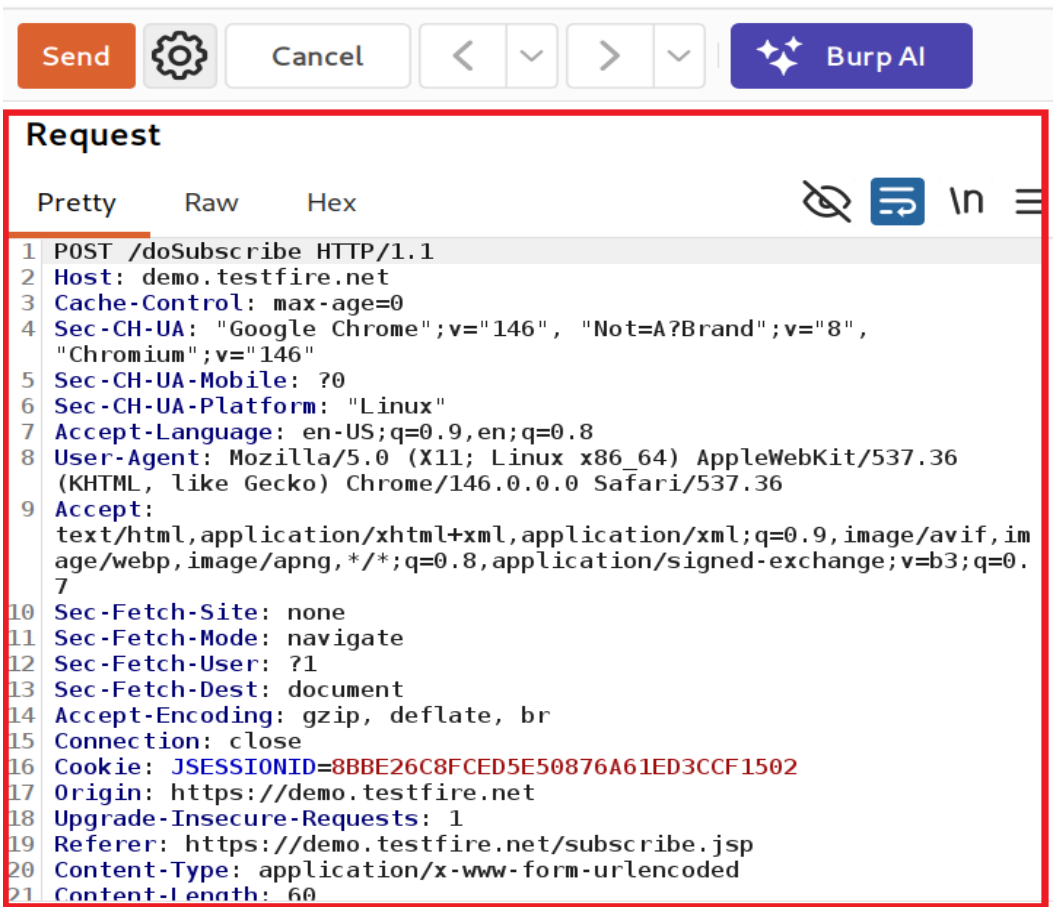
Exploitation Details

During the security assessment, it was identified that sensitive application functions do not properly validate request authenticity. This weakness may allow attackers to craft malicious requests that execute actions using the victim's authenticated session.

Used Tools

- Burp Suite Pro
- OWASP ZAP

Proof of Concept Screenshot



Resolution

- Implement anti-CSRF tokens for all state-changing requests.
- Use SameSite cookie attributes.
- Verify the Origin and Referer headers where applicable.
- Require re-authentication for sensitive actions.
- Avoid using GET requests for sensitive operations.

2.5 No Rate Limit

No Rate Limit vulnerability occurs when the application does not properly restrict the number of requests a user or client can send within a specific period.

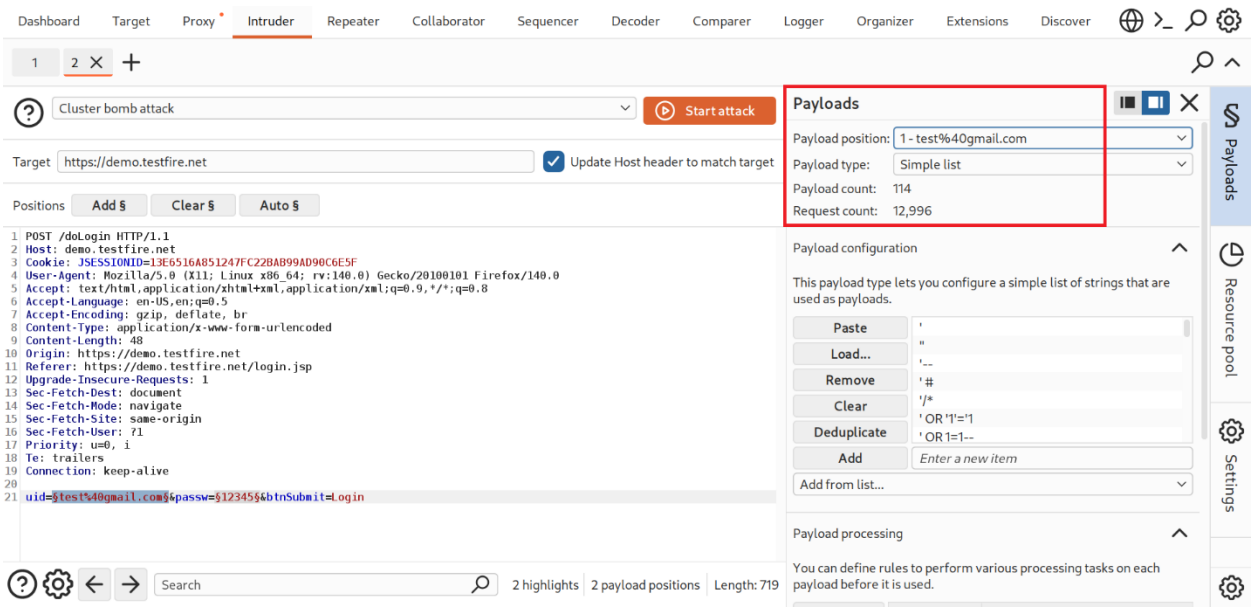
Exploitation Details

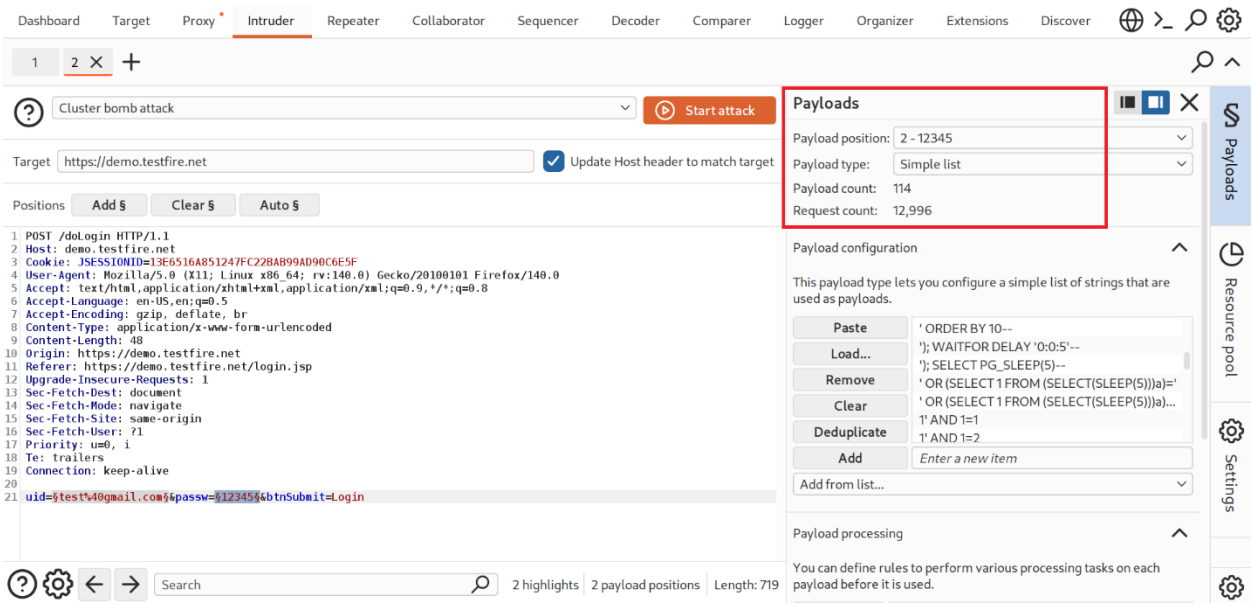
During the penetration testing assessment, it was identified that the application accepts unlimited requests without applying proper request throttling controls. This weakness may allow attackers to abuse authentication, OTP, API, or other sensitive functionalities.

Used Tools

- Burp Suite Pro
- OWASP ZAP

Proof of Concept Screenshot





Resolution

- Implement request rate limiting on sensitive endpoints.
- Apply account/IP-based throttling mechanisms.
- Introduce CAPTCHA protection where appropriate.
- Monitor abnormal request patterns and automated activities.
- Configure temporary blocking for excessive requests.

2.6 Open Redirection (DOM-based)

DOM-based Open Redirection is a client-side vulnerability where user-controlled input is used to redirect users to external or malicious destinations without proper validation.

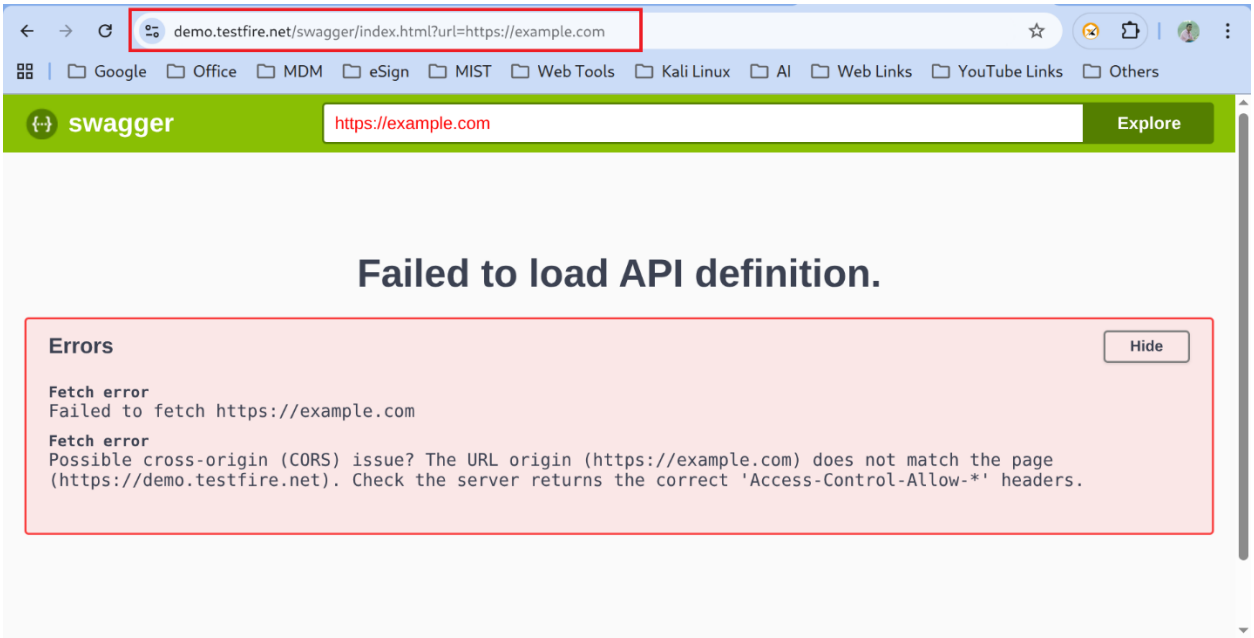
Exploitation Details

During the security assessment, it was identified that the application processes untrusted input within client-side scripts to perform URL redirection. This weakness may allow attackers to redirect users to malicious websites for phishing or malware distribution purposes.

Used Tools

- Burp Suite Pro
- OWASP ZAP
- Browser Developer Tools

Proof of Concept Screenshot



Resolution

- Avoid using user-controlled input directly in redirection logic.
- Validate and whitelist trusted redirect destinations.
- Implement proper client-side input validation.
- Use relative paths instead of absolute URLs where possible.
- Monitor and restrict unsafe JavaScript redirection methods.

2.7 Frameable Response (Potential Clickjacking)

Frameable Response vulnerability occurs when application pages can be embedded within external frames or iframes, potentially enabling clickjacking attacks.

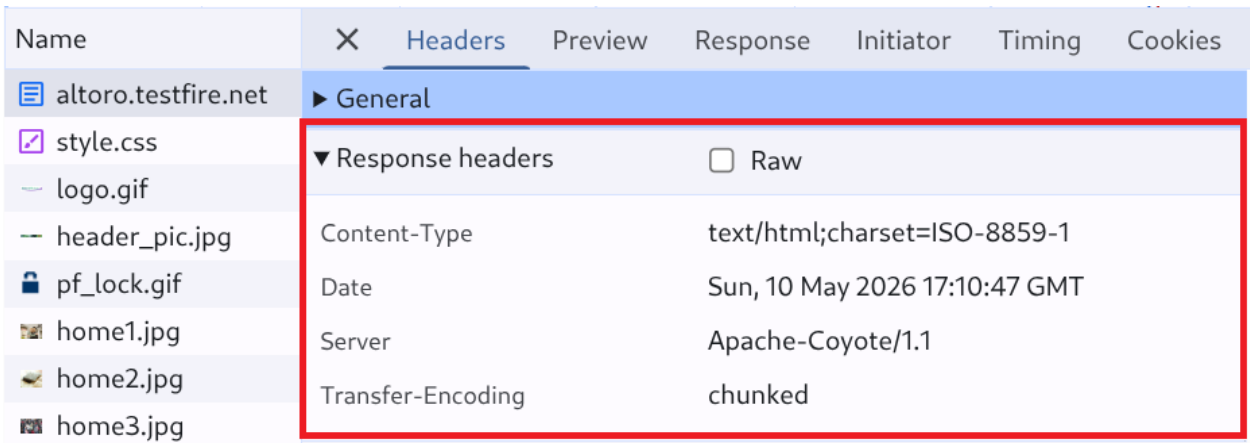
Exploitation Details

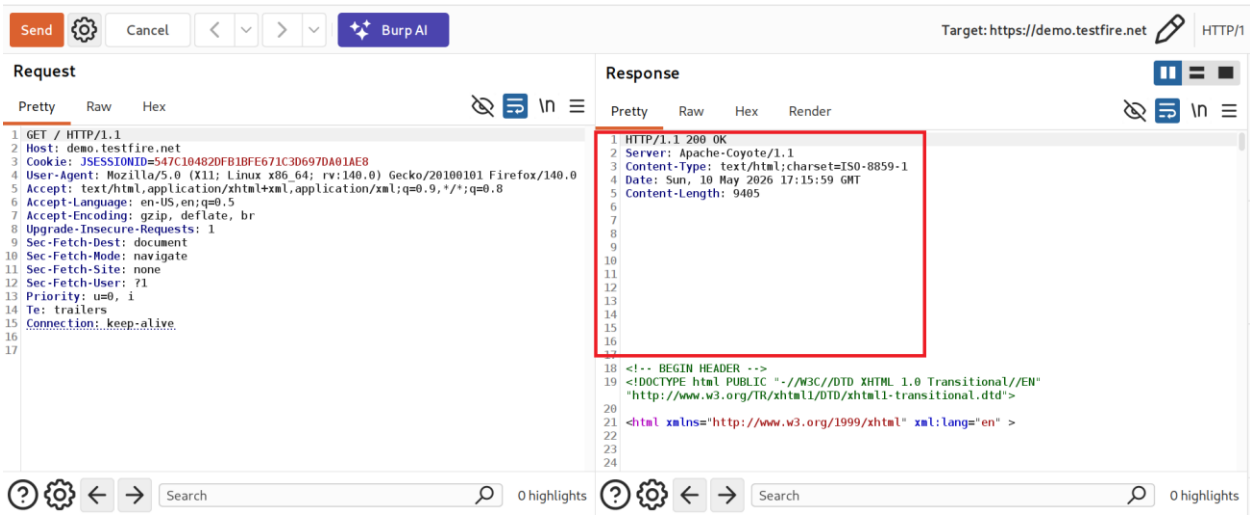
During the penetration testing assessment, it was identified that the application does not implement proper anti-framing protections. This weakness may allow attackers to trick users into performing unintended actions through malicious overlay techniques.

Used Tools

- Burp Suite Pro
- OWASP ZAP
- Browser Developer Tools

Proof of Concept Screenshot





Resolution

- Implement the X-Frame-Options header with DENY or SAMEORIGIN policy.
- Apply Content Security Policy (CSP) frame-ancestors directive.
- Prevent sensitive pages from being embedded in external frames.
- Regularly test application pages against clickjacking attacks.

2.8 Content Security Policy (CSP) Issues

Content Security Policy (CSP) Issues occur when the application does not properly define or enforce CSP headers to restrict unsafe resource loading and script execution.

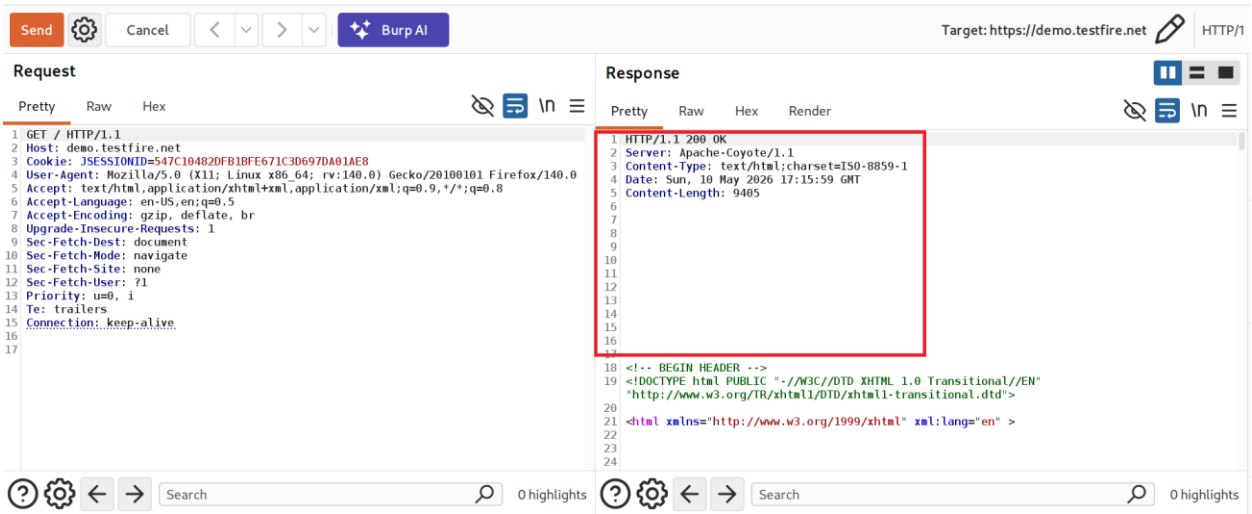
Exploitation Details

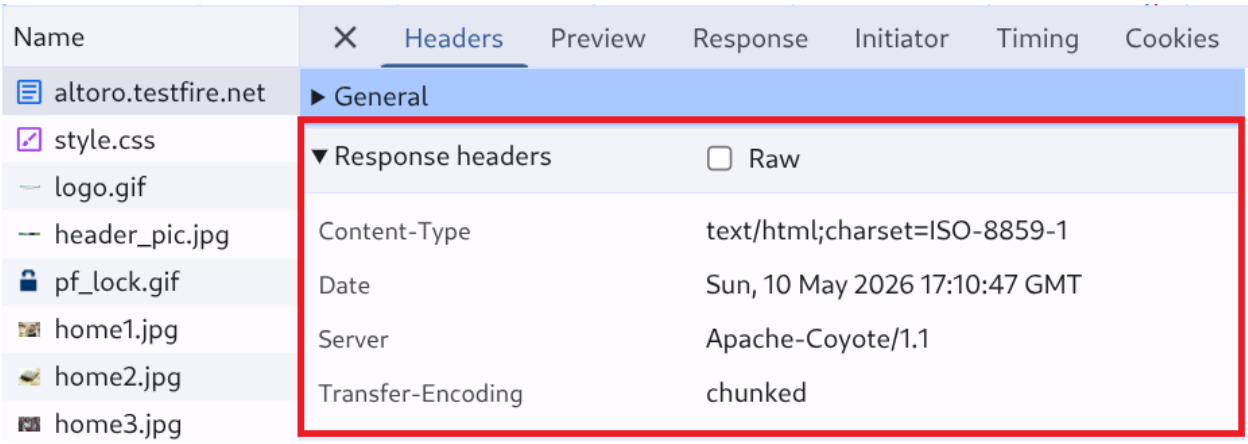
During the security assessment, it was identified that the application either lacks CSP headers or implements weak CSP configurations. This weakness may increase the risk of XSS attacks and unauthorized resource loading.

Used Tools

- Burp Suite Pro
- OWASP ZAP
- Browser Developer Tools
- Acunetix

Proof of Concept Screenshot





Resolution

- Implement a strong Content Security Policy header.
- Restrict script execution to trusted domains only.
- Avoid using unsafe-inline and unsafe-eval directives.
- Regularly review and update CSP configurations.
- Monitor CSP violation reports for suspicious activities.

2.9 Strict Transport Security (HSTS) Not Enforced

HSTS Not Enforced vulnerability occurs when the application does not instruct browsers to enforce secure HTTPS connections.

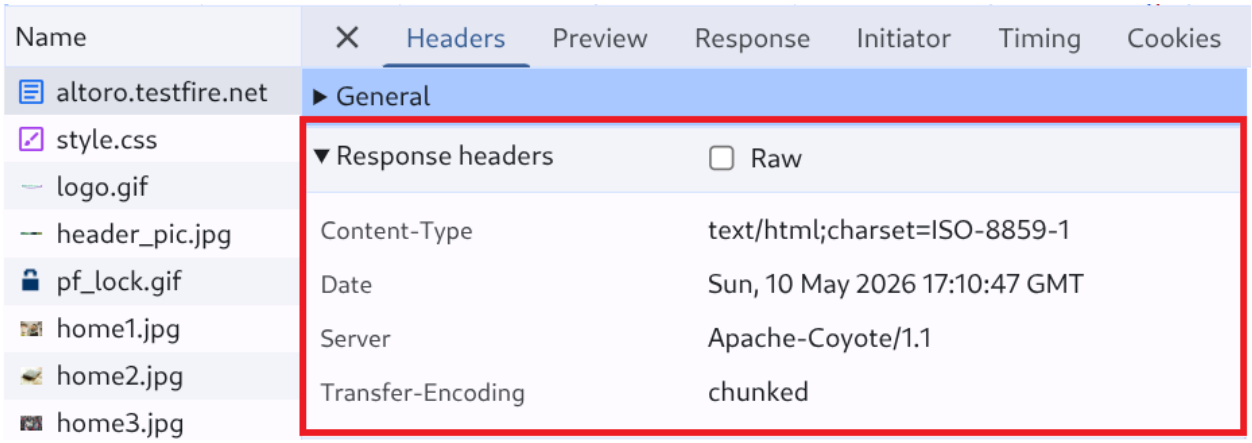
Exploitation Details

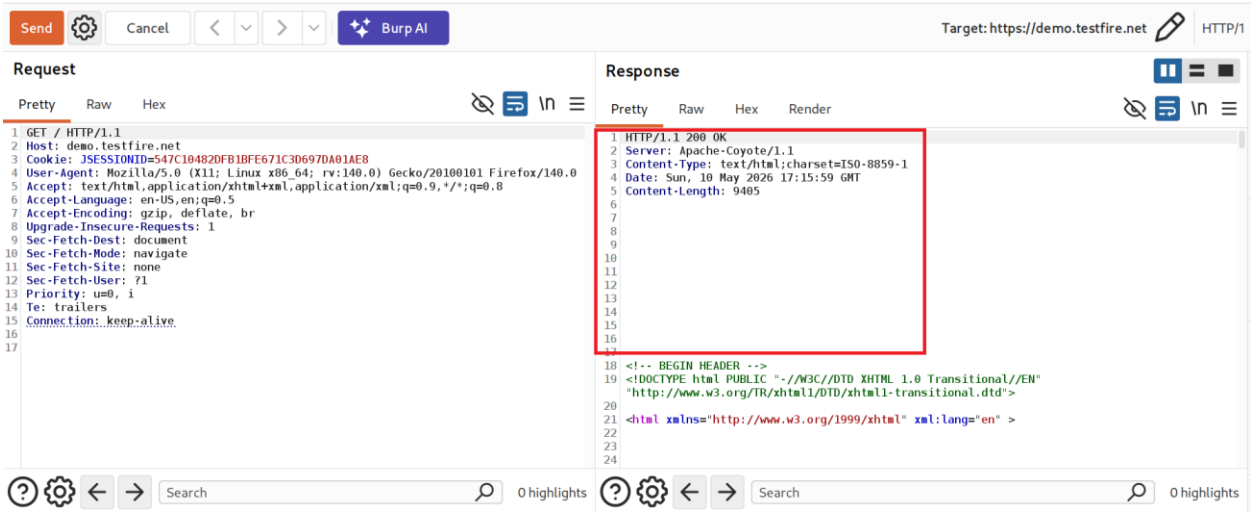
During the penetration testing assessment, it was identified that the application does not implement the Strict-Transport-Security header. This weakness may expose users to SSL stripping and protocol downgrade attacks.

Used Tools

- Burp Suite Pro
- OWASP ZAP
- Browser Developer Tools

Proof of Concept Screenshot





Resolution

- Implement the Strict-Transport-Security header.
- Enforce HTTPS connections across the entire application.
- Configure an appropriate max-age value for HSTS.
- Include subdomains where applicable.
- Redirect all HTTP traffic to HTTPS.

3 Informational Issues and Remediation Suggestion

Cacheable HTTPS Response

Cacheable HTTPS Response vulnerability occurs when sensitive HTTPS responses are improperly configured to allow browser or intermediary caching.

Exploitation Details

During the security assessment, it was identified that sensitive application responses may be cached by browsers or proxy servers. This weakness may lead to unauthorized disclosure of sensitive information from cached content.

Resolution

- Configure proper Cache-Control and Pragma headers.
- Disable caching for sensitive pages and responses.
- Use no-store and no-cache directives where appropriate.
- Prevent sensitive information from being stored in shared caches.
- Regularly review response header configurations.

DOM Data Manipulation (DOM-based)

DOM Data Manipulation vulnerability occurs when client-side scripts improperly process user-controlled data within the Document Object Model (DOM).

Exploitation Details

During the penetration testing assessment, it was identified that the application processes untrusted data directly within DOM operations. This weakness may allow attackers to manipulate page content or trigger unintended client-side behavior.

Resolution

- Avoid directly inserting untrusted data into DOM elements.
- Implement proper input validation and output encoding.
- Use secure JavaScript APIs for DOM manipulation.
- Sanitize all user-controlled client-side data.
- Conduct regular DOM security testing.

Cross-domain Referrer Leakage

Cross-domain Referrer Leakage occurs when sensitive information is unintentionally exposed through HTTP Referrer headers to external domains.

Exploitation Details

During the security assessment, it was identified that the application may disclose internal URLs or sensitive parameters through Referrer headers during cross-domain requests. This weakness may expose sensitive information to third parties.

Resolution

- Implement a strict Referrer-Policy header.
- Avoid exposing sensitive information in URLs.
- Restrict unnecessary cross-domain requests.
- Use secure communication mechanisms for sensitive data transfer.
- Regularly review application header configurations.

Path-relative Style Sheet Import

Path-relative Style Sheet Import vulnerability occurs when relative stylesheet paths are improperly resolved, potentially allowing attackers to manipulate imported resources.

Exploitation Details

During the penetration testing assessment, it was identified that the application uses path-relative stylesheet imports without proper handling. This weakness may lead to unintended stylesheet loading behavior and content injection risks.

Resolution

- Use absolute paths for stylesheet imports.
- Properly define the document base URL.
- Validate and sanitize dynamically generated paths.
- Avoid ambiguous relative path configurations.
- Regularly review client-side resource loading behavior.

Referrer-dependent Response

Referrer-dependent Response vulnerability occurs when application behavior changes based on the HTTP Referrer header without proper validation.

Exploitation Details

During the security assessment, it was identified that certain application responses rely on the Referrer header value. This weakness may allow attackers to manipulate application behavior through forged or modified headers.

Resolution

- Avoid relying solely on the Referrer header for security decisions.
- Implement proper server-side validation mechanisms.
- Use secure session and authorization controls.
- Validate request origin using multiple security checks.
- Monitor suspicious or malformed request headers.

Input Returned in Response (Reflected)

Input Returned in Response vulnerability occurs when user-supplied input is reflected back within server responses without proper sanitization.

Exploitation Details

During the penetration testing assessment, it was identified that application responses include user-controlled input directly within returned content. This weakness may increase the risk of reflected XSS and information disclosure issues.

Resolution

- Properly sanitize and encode user-supplied input before reflecting it in responses.
- Implement strict output encoding mechanisms.
- Avoid unnecessary reflection of user-controlled data.
- Apply secure input validation controls.
- Conduct regular testing for reflected injection issues.

Sensitive Data Exposure (Email Address)

Sensitive Data Exposure vulnerability occurs when sensitive information such as email addresses is unnecessarily disclosed within application responses or publicly accessible content.

Exploitation Details

During the security assessment, it was identified that email addresses are exposed within application responses or publicly accessible resources. This weakness may increase the risk of phishing, spam, or targeted attacks.

Resolution

- Avoid exposing sensitive user information unnecessarily.
- Mask or partially obfuscate sensitive data where possible.
- Restrict public access to sensitive resources.
- Implement proper access control mechanisms.
- Regularly review application responses for sensitive data exposure.

HTML Does Not Specify Charset

HTML Does Not Specify Charset vulnerability occurs when web pages fail to explicitly define the character encoding standard.

Exploitation Details

During the penetration testing assessment, it was identified that the application does not specify a charset within HTML responses. This weakness may lead to inconsistent browser interpretation and potential security issues.

Resolution

- Explicitly define charset using the Content-Type header or meta charset tag.
- Use UTF-8 encoding consistently across the application.
- Configure web servers to enforce proper character encoding.
- Regularly validate HTML response headers and metadata.

Certificate Observation

TLS Certificate Observation refers to informational findings related to TLS/SSL certificate configuration, validity, or deployment details.

Exploitation Details

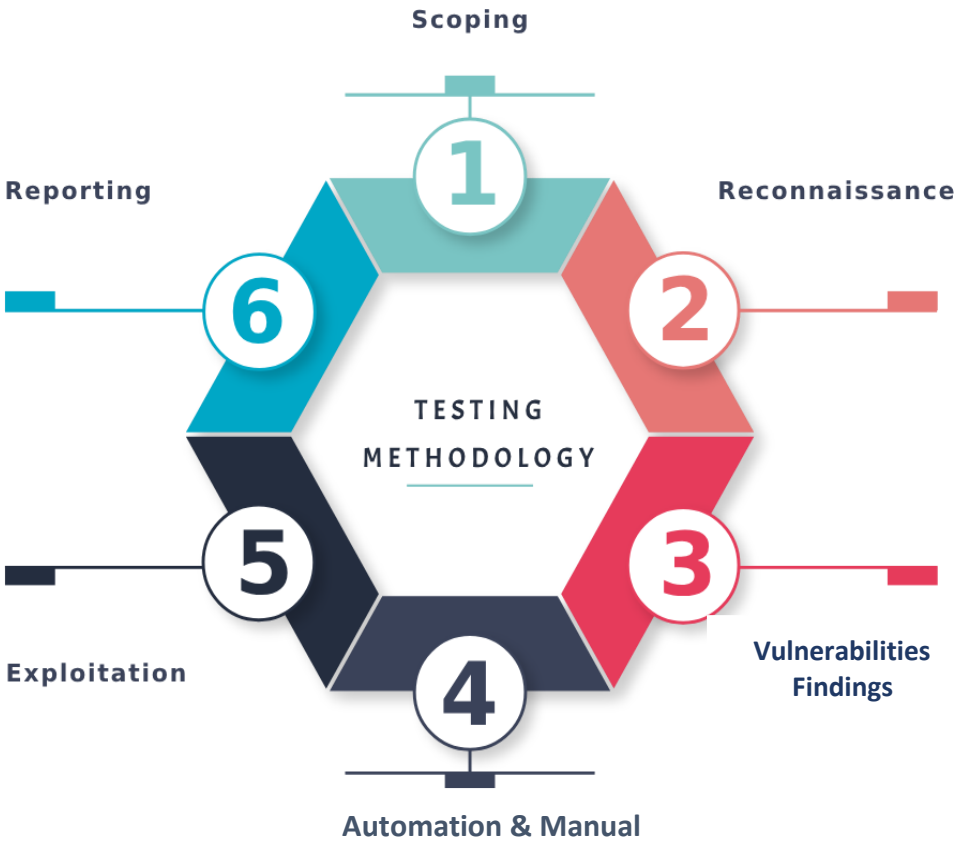
During the security assessment, observations related to TLS certificate configuration were identified. These findings may include certificate validity details, issuer information, or configuration-related observations that should be reviewed for security best practices.

Resolution

- Regularly monitor TLS/SSL certificate validity and expiration dates.
- Use certificates issued by trusted Certificate Authorities (CA).
- Disable weak SSL/TLS protocols and cipher suites.
- Ensure proper certificate chain configuration.
- Conduct periodic TLS security assessments.

4.0 Testing Methodology

The testing methodology followed during the Web Penetration Testing (PT) adheres to industry best practices and consists of several key phases. This section provides an overview of the testing methodology employed during the assessment, applicable to various organizations and scenarios.



Scoping:
This phase includes identifying the target web applications, systems, APIs, and infrastructure components to be assessed, along with defining the scope limitations, exclusions, and testing boundaries.

Reconnaissance:
• The reconnaissance phase focuses on gathering information about the target organization, its systems, and potential attack surfaces.
• Both passive and active reconnaissance techniques are utilized to identify publicly available information, network infrastructure, technologies, and potential entry points.

Vulnerability Identification:
• This phase involves identifying known vulnerabilities, security weaknesses, and misconfigurations within the target web applications and associated systems.
• The assessment includes security checks related to authentication, input validation, session management, access controls, and security configurations.

Automation & Manual Testing:
• Automated vulnerability scanning tools are utilized to identify common security weaknesses and misconfigurations.
• Manual testing is conducted to validate identified findings and discover vulnerabilities that may not be detected through automated scanning.
• This phase involves detailed testing of application functionalities, authentication mechanisms, authorization controls, session handling, and client-side security.

Exploitation:
• The exploitation phase focuses on securely validating identified vulnerabilities to determine their potential impact and confirm their existence.
• Controlled exploitation techniques are performed while minimizing any operational impact on the target environment.

Reporting:
• The reporting phase involves documenting the identified findings, associated risks, affected components, and remediation recommendations.
• The report provides a clear and concise overview of the assessment results and the overall security posture of the application.

5.0 Testing Methodology

During the Penetration Testing (PT), the identified vulnerabilities and findings are categorized into the following risk levels:

Critical:
Critical vulnerabilities pose an immediate and severe threat to the security of the application and associated systems. Exploitation of these vulnerabilities may result in complete system compromise, unauthorized access to sensitive information, or significant financial and reputational damage.

High-Risk:
High-risk vulnerabilities represent significant security weaknesses that could potentially lead to unauthorized access, data breaches, privilege escalation, or compromise of sensitive information.

Medium-Risk:
Medium-risk vulnerabilities indicate security weaknesses that may be exploited under specific conditions. Successful exploitation could result in limited unauthorized access, information disclosure, or partial compromise of application functionalities.

Low-Risk:
Low-risk vulnerabilities represent minor security weaknesses or security hardening gaps that are less likely to result in significant impact but may increase the overall attack surface of the application if left unresolved.

Informational:
Informational findings represent security observations, missing best practices, or configuration-related recommendations that do not pose a direct exploitation risk but may help improve the overall security posture of the application.

6.0 Conclusion

The Web Penetration Testing (PT) assessment conducted against the target application identified multiple security vulnerabilities and configuration-related weaknesses across different components of the application environment. The assessment revealed issues related to input validation, authentication and authorization mechanisms, session management, client-side security controls, security header implementation, and overall application hardening practices.

During the assessment, vulnerabilities of varying severity levels, including High, Medium, Low, and Informational findings, were identified and validated through a combination of automated and manual testing methodologies. Several identified weaknesses may potentially allow attackers to perform unauthorized actions, gain access to sensitive information, abuse application functionalities, or compromise user accounts if exploited successfully.

The assessment also identified multiple security hardening and configuration-related observations that, while not directly exploitable, may increase the overall attack surface of the application and reduce the effectiveness of existing security controls. Addressing these weaknesses will help improve the application's overall resilience against common web-based attacks and emerging cyber threats.

Overall, the penetration testing exercise provided valuable insight into the current security posture of the application and highlighted several areas requiring remediation and security improvements. It is strongly recommended that the identified findings be remediated based on their associated risk levels and business impact. Implementing secure coding practices, proper input validation, strong authentication controls, security header enforcement, rate limiting mechanisms, and continuous security monitoring will significantly improve the security posture of the application and reduce potential risks to the organization.